



NETWORK SECURITY | MODULE 7

Modern Block Ciphers and Modes of Operation

Lesson 13 – IDEA

Lesson 14 – Advanced Encryption Standard

Lesson 15 – Modes of Operation



AES state

Add* · Multiply* · 52 sub-keys · SubBytes · ShiftRows · MixColumns · ECB · CBC · CFB · OFB



LESSON 13: IDEA

1

IDEA Basics

64-bit blocks, 128-bit key, three operations

2

Rounds and Sub-keys

8 rounds, 52 sub-keys, key shifting

3

Output and Strength

Final transformation and security

LESSON 14: AES

4

AES Overview

Block, key sizes, the state

5

Four Transformations

SubBytes, ShiftRows, MixColumns, AddRoundKey

6

Key Expansion

Rationale and the equivalent inverse cipher

LESSON 15: MODES

7

ECB and CBC

Block-by-block and chaining

8

CFB and OFB

Block ciphers used as stream ciphers

9

Comparing the Modes

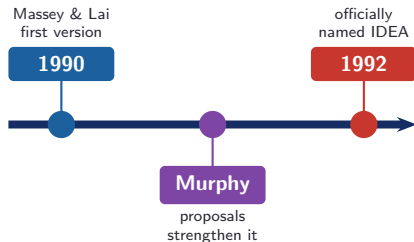
Which mode for which job



Lesson 13

IDEA

IDEA: International Data Encryption Algorithm

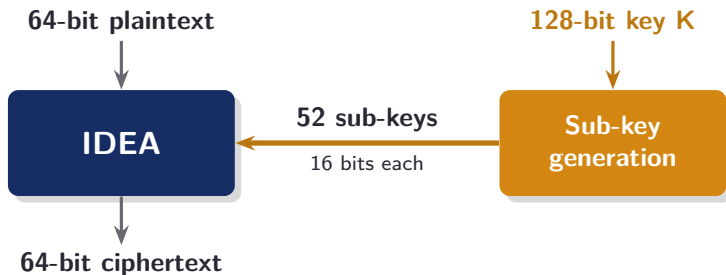


- Encrypts **64-bit** blocks using a **128-bit** key
- Based on three algebraic groups: **XOR**, **addition modulo 2^{16}** and **multiplication modulo $2^{16} + 1$**
- Unlike DES: **no permutations, no S-boxes**
- Used in the public-domain **Pretty Good Privacy (PGP)** e-mail system

Why it was trusted

Secure against **differential cryptanalysis**, and its key is too long for exhaustive search.

Block Diagram of IDEA



Basic principles

IDEA is a **symmetric block cipher**. Like DES it works on 64-bit blocks and is **reversible** (the same algorithm encrypts and decrypts), and uses both **diffusion and confusion**. But its round function and sub-key generation are very different from DES.

The Three Operations of IDEA



XOR

Bitwise exclusive-OR
of two 16-bit values



Add*

Addition **mod**-
ulo $2^{16} = 65536$



Multiply*

Multiplication **mod**-
ulo $2^{16} + 1 = 65537$

Why the asterisk?

Add* and Multiply* are **not ordinary** addition and multiplication. Results are reduced with **mod** so they always fit in 16 bits. ($a \bmod b$ is the remainder of a/b : $5 \bmod 2 = 1$, $5 \bmod 3 = 2$.)

Why Modulo? A Worked Example (Fig. 13.4)



$$\begin{array}{r} + \quad 1111111100000000 \\ \quad 1111111111000001 \\ \hline 11111111011000001 \quad 17 \text{ bits!} \end{array}$$

$$130753 \bmod 65536 = 65217$$

↓

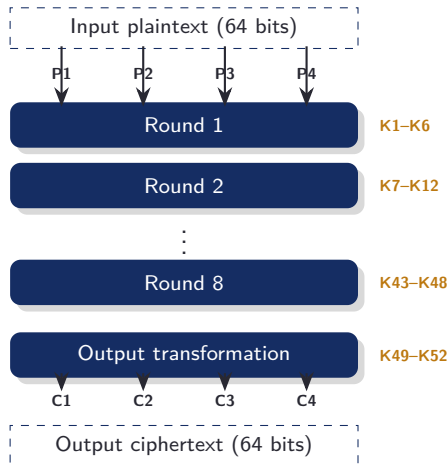
$$11111111011000001 \quad 16 \text{ bits}$$

- In round 2, suppose $P_2 = 1111111100000000$ and $K_2 = 1111111111000001$
- Ordinary addition gives a **17-bit** result (130753)
- Only **16 bit positions** are available
- Taking mod 65536 gives 65217, which fits in 16 bits

So

Modulo arithmetic brings every result **back to 16 bits**.

Broad-Level Steps (Fig. 13.2)



- 1 The 64-bit block is split into four **16-bit** parts, P1–P4
- 2 There are **8 rounds**; each uses **6 sub-keys**
- 3 Round 1 uses K1–K6, round 2 uses K7–K12, . . . , round 8 uses K43–K48
- 4 A final **output transformation** uses 4 sub-keys, K49–K52
- 5 Its output, C1–C4, forms the 64-bit ciphertext

Count

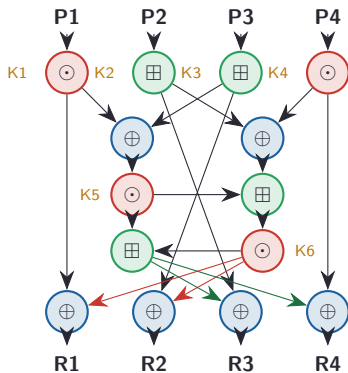
$$8 \times 6 + 4 = \mathbf{52 \text{ sub-keys}}$$

The 14 Steps of One Round (Fig. 13.3)



Steps 1–4 mix the four inputs with K1–K4; steps 5–10 form the “MA” structure with K5 and K6; steps 11–14 give R1–R4. **Multiply***, **Add***, **XOR**.

One Round of IDEA (Fig. 13.5)



- Same steps as before, drawn **symbolically**
- Inputs P1–P4, sub-keys K1–K6, outputs **R1–R4**
- R1–R4 are **not** the final ciphertext: they feed the next round, or the output transformation
- Note R2 and R3 are **swapped** in the middle blocks

Legend

⊙ Multiply* ⊞ Add* ⊕ XOR

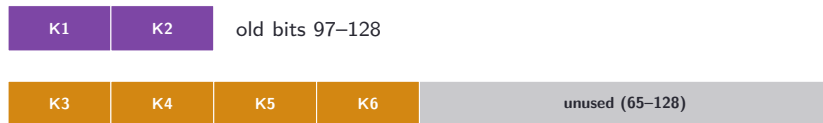
Sub-Key Generation: Rounds 1 and 2



Round 1: original 128-bit key



Round 2: bits 97–128, then shift key left by 25



first 64 bits of the **shifted** key

- Round 1 uses bits **1–96** (6 sub-keys $\times$ 16 bits); bits 97–128 are left over
- Round 2 needs 96 bits: 32 left over + 64 more. The key is **circular-left shifted by 25 bits**: bit 26 becomes bit 1, bit 25 becomes bit 128

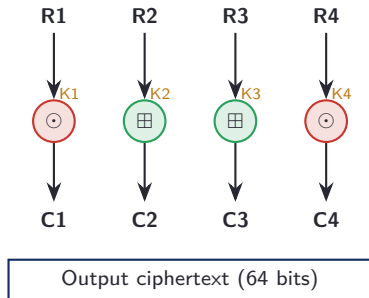
Sub-Key Generation for All Rounds (Table 13.2)



Round	Sub-keys from current key	Then	Left over
1, 5	Bits 1–96 $\rightarrow$ K1–K6	—	97–128
2, 6	Bits 97–128 $\rightarrow$ K1, K2	Shift 25; bits 1–64 $\rightarrow$ K3–K6	65–128
3, 7	Bits 65–128 $\rightarrow$ K1–K4	Shift 25; bits 1–32 $\rightarrow$ K5, K6	33–128
4, 8	Bits 33–128 $\rightarrow$ K1–K6	Shift 25	none
Output	Bits 1–64 $\rightarrow$ K1–K4	—	—

Every round takes **96 bits** this way. The pattern repeats every 4 rounds, and at the end of round 8 **no bits are unused**.

Output Transformation (Figs. 13.8 and 13.9)



- A **one-time** operation after round 8
- Input: round 8's output, R1–R4 (four 16-bit blocks)
- Uses **4 sub-keys**, not 6

Four steps

1. Multiply* R1 and K1
2. Add* R2 and K2
3. Add* R3 and K3
4. Multiply* R4 and K4

Its sub-keys: after round 8 the key is shifted by 25 again, and the **first 64 bits** become K1–K4.



Decryption

Exactly the **same process** as encryption, with changes in the generation and pattern of sub-keys: the decryption sub-keys are the **inverses** of the encryption sub-keys.



Strength

A **128-bit** key, double the size of DES. Brute force needs $2^{128} \approx 10^{38}$ operations.

 DES: 56 bits

 IDEA: 128 bits

At one encryption per microsecond, trying half the keys would take more than 5.4×10^{24} **years!**



1

A strong encryption algorithm

2

Uses simple binary operations: XOR, ADD, MULTIPLY

3

Hard to cryptanalyse because of its rounds and operations

4

Efficient in both software and hardware

5

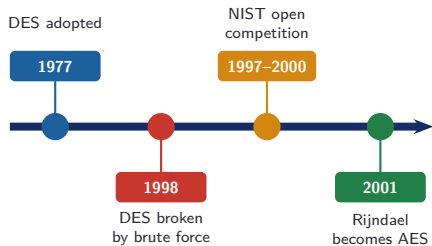
Used in Pretty Good Privacy (PGP)



Lesson 14

Advanced Encryption Standard

Why a New Standard?



- DES's **56-bit key** became too short; triple DES was secure but **slow**
- NIST ran an open competition for a replacement
- The winner, **Rijndael** (Daemen and Rijmen), became the **Advanced Encryption Standard (AES)**
- Being **royalty-free** was a key selection criterion (recall Module 5)



Key size	128 bits	192 bits	256 bits
Block size	128 bits	128 bits	128 bits
Number of rounds	10	12	14
Round key size	128 bits	128 bits	128 bits
Expanded key	44 words	52 words	60 words

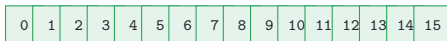
Compare with DES

Twice the block size, and at least 2^{72} times more keys. AES is **not a Feistel cipher**: every round transforms the **whole block**, not just half.

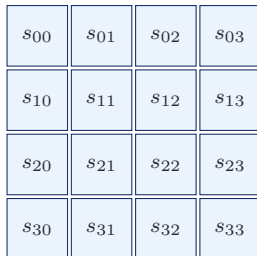
The State: AES Thinks in a 4×4 Grid



16 input bytes

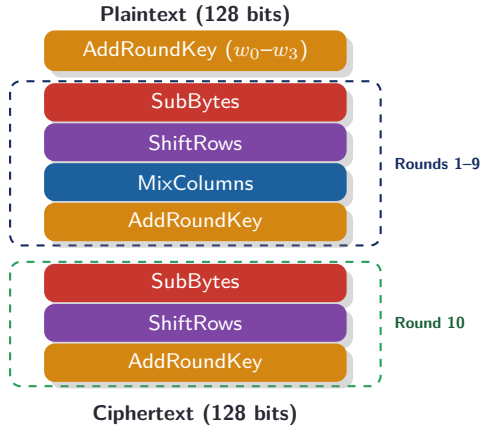


filled column by column



- The 128-bit block (16 bytes) is copied into a 4×4 array of bytes called the **State**
- Bytes fill the State **column by column**
- Every transformation changes the State; at the end it is copied out as the ciphertext
- The key is also arranged as a matrix of bytes and expanded into **words** (4 bytes each)

AES Cipher: Overall Structure



- Begins with an **AddRoundKey** stage
- Then N rounds; for a 128-bit key, $N = 10$
- Each full round has **four stages**
- The **last round omits MixColumns**

One permutation, three substitutions

ShiftRows is a permutation; SubBytes, MixColumns and AddRoundKey are substitutions. Only **AddRoundKey** uses the key.

The Four Transformations at a Glance



SubBytes

Each byte is replaced using a fixed **S-box** lookup table (non-linear)

ShiftRows

Each row is **rotated left** by its row number (0, 1, 2, 3)

MixColumns

Each column is **multiplied by a fixed matrix** in $GF(2^8)$

AddRoundKey

The State is **XORed** with the 128-bit round key

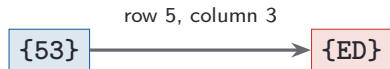
Each has a **forward** form (for encryption) and an **inverse** form (for decryption).



EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

S-box →

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6



Forward transformation

A simple **table lookup**. AES defines a 16×16 S-box of byte values. For each State byte, the **left 4 bits** pick the row and the **right 4 bits** pick the column.

Inverse transformation

Uses the **inverse S-box**: InvSubBytes maps {ED} back to {53}.



Low correlation

Little correlation between input bits and output bits

No fixed points

No byte maps to itself, and none to its complement

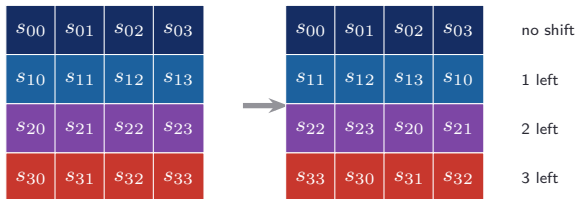
Non-linear

Built from the multiplicative inverse in $GF(2^8)$, so it resists linear and differential cryptanalysis

Simple to describe

One table, yet designed from clear mathematics

The S-box is the **only non-linear** step in AES, just as the S-boxes were in DES.



Forward ShiftRows

Row 0 is not changed. Row 1 is rotated **1 byte left**, row 2 **2 bytes**, row 3 **3 bytes**.

Inverse ShiftRows

The same shifts, but to the **right**.

Rationale

The 4 bytes of one column are **spread to four different columns**. Without this, AES would be four independent ciphers.



$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0j} \\ s_{1j} \\ s_{2j} \\ s_{3j} \end{bmatrix} = \begin{bmatrix} s'_{0j} \\ s'_{1j} \\ s'_{2j} \\ s'_{3j} \end{bmatrix}$$

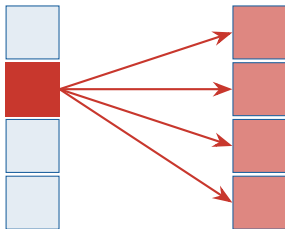
$$s'_{0j} = (2 \cdot s_{0j}) \oplus (3 \cdot s_{1j}) \oplus s_{2j} \oplus s_{3j}$$

Forward MixColumns

Works on each **column** separately. Each byte is replaced by a function of **all four bytes** in its column: a matrix multiplication in $GF(2^8)$, where “+” is XOR.

Inverse MixColumns

Multiply by the inverse matrix, with entries {0E}, {0B}, {0D}, {09}.



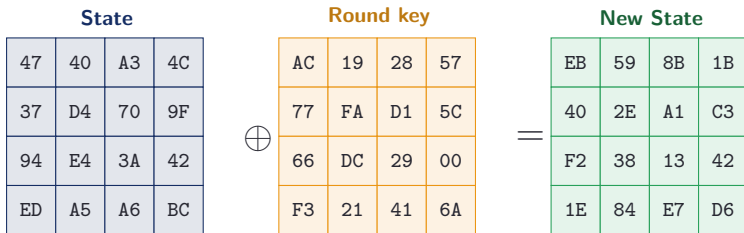
1 changed input byte
⇒ all 4 output bytes change

- Gives good **mixing among the bytes** of each column
- Combined with ShiftRows, after a few rounds **every output bit depends on every input bit**
- Coefficients $\{01\}$, $\{02\}$, $\{03\}$ make it fast: multiplying by 2 is a shift and a conditional XOR

Why skip it in the last round?

It makes the cipher and its inverse look alike, and adds no security at the very end.

AddRoundKey Transformation

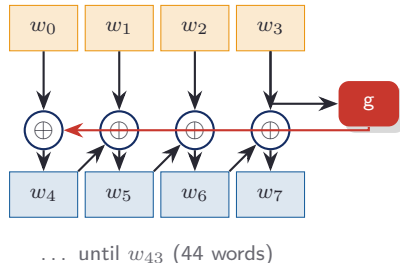


Forward and inverse

The 128 bits of the State are **XORed** with the 128 bits of the round key. XOR is its own inverse, so the inverse is the **same operation**.

Rationale

As simple as possible, yet it affects **every bit**. The key expansion's complexity plus the other stages give the security.



- The 16-byte key fills the first 4 words w_0-w_3
- Each new word: $w_i = w_{i-1} \oplus w_{i-4}$
- For every **4th** word, w_{i-1} first goes through **g**

The function g

1. **RotWord**: rotate the 4 bytes left by one
2. **SubWord**: S-box on each byte
3. XOR with a round constant **Rcon**: 01, 02, 04, 08, 10, 20, 40, 80, 1B, 36



1

Knowing part of the key or round key does **not** reveal the rest

2

An **invertible** transformation: any 4 consecutive words give back the whole schedule

3

Fast on a wide range of processors

4

Round constants **remove symmetry** between rounds

5

Each key bit **diffuses** into many round-key bits

6

Enough **non-linearity** to stop round-key differences being fully predictable

Design goal

Resist known attacks, especially attacks where the attacker knows or can control **part of the key**.



Encryption round

SubBytes

ShiftRows

MixColumns

AddRoundKey

Straightforward decryption

InvShiftRows

InvSubBytes

AddRoundKey

InvMixColumns

the order of steps is **different**, so the two cannot share the same structure

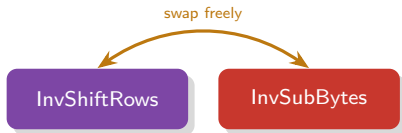
- The straightforward decryption cipher uses the inverse steps in a **different order**
- For efficiency, we want decryption to have the **same structure** as encryption
- Two swaps make this possible, giving the **equivalent inverse cipher**

Benefit

The same hardware or code layout can be used for both directions.

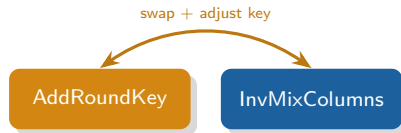


InvShiftRows $\leftrightarrow$ InvSubBytes



InvShiftRows moves bytes **without changing** them; InvSubBytes changes bytes **without moving** them. So their order does not matter.

AddRoundKey $\leftrightarrow$ InvMixColumns



InvMixColumns is **linear**:
$$\text{InvMix}(S \oplus K) = \text{InvMix}(S) \oplus \text{InvMix}(K).$$
 So swap them and apply InvMixColumns to the **round key** as well.

Result

Decryption becomes: InvSubBytes, InvShiftRows, InvMixColumns, AddRoundKey with modified keys: the **same layout** as encryption.



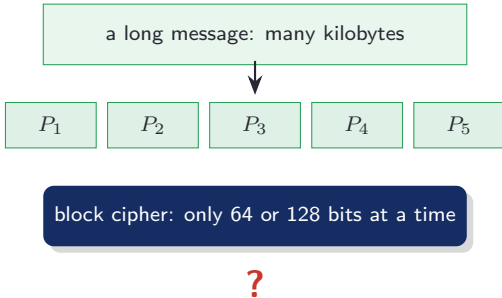
Feature	DES	IDEA	AES
Block size	64 bits	64 bits	128 bits
Key size	56 bits	128 bits	128 / 192 / 256 bits
Rounds	16	8 + output transf.	10 / 12 / 14
Structure	Feistel	Mixed algebraic groups	Substitution–permutation
Non-linearity from	8 S-boxes	Multiply* mod $2^{16}+1$	One byte S-box
Status	Broken by brute force	Used in PGP	Current standard



Lesson 15

Modes of Operation

Why Do We Need Modes of Operation?

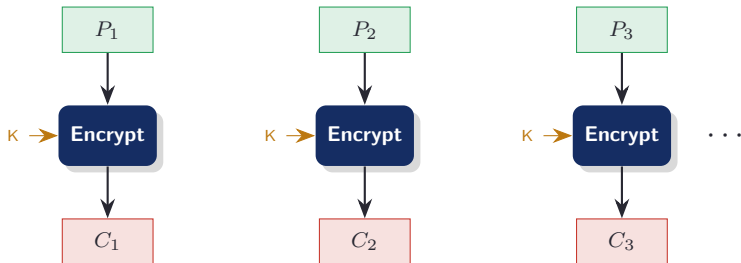


- A block cipher encrypts **one fixed-size block**
- Real messages are much **longer**
- A **mode of operation** says how to apply the cipher to a sequence of blocks
- The last block may need **padding** to fill it

Four block cipher modes

ECB, CBC, CFB, OFB

Electronic Code Book (ECB) Mode



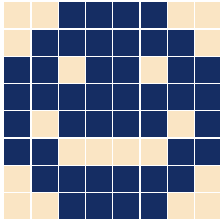
How it works

Each block is encrypted **independently** with the same key: $C_i = E_K(P_i)$. Like a code book: every plaintext block has its own ciphertext entry.

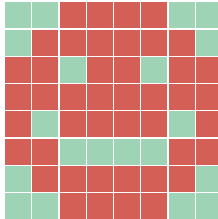
Weakness

Identical plaintext blocks give identical ciphertext blocks, so patterns show through. Good only for short data, such as a single key.

Plaintext image

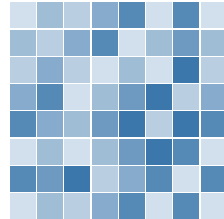


ECB ciphertext



same block $\Rightarrow$
same output: the
shape is still visible

CBC ciphertext

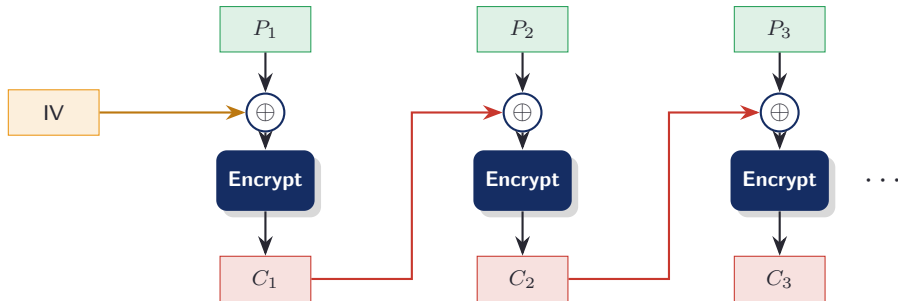


chaining hides the
pattern: looks **random**

Each small square stands for one block. ECB turns each distinct block into its own distinct colour, so the picture survives encryption.



Cipher Block Chaining (CBC) Mode



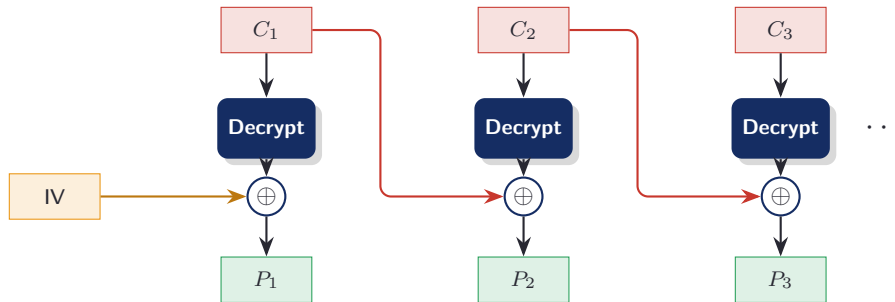
How it works

Each plaintext block is **XORed with the previous ciphertext block** before encryption:
 $C_i = E_K(P_i \oplus C_{i-1})$, with $C_0 =$ an **Initialization Vector (IV)**.

Benefit

Identical plaintext blocks now give **different** ciphertext. Decryption: $P_i = D_K(C_i) \oplus C_{i-1}$. Widely used for bulk data and authentication.

CBC Decryption



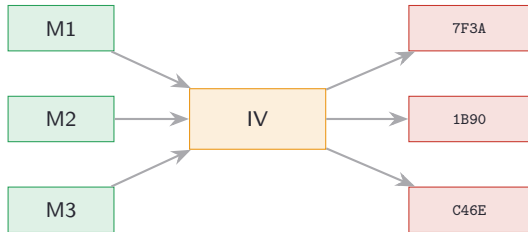
Formula

$P_i = D_K(C_i) \oplus C_{i-1}$, with $C_0 = IV$. Each block is decrypted, then XORed with the **previous ciphertext** block.

Error propagation

A bit error in C_i garbles all of P_i and flips **one bit** of P_{i+1} . After that, decryption recovers.

The Initialization Vector (IV)



same message, **different IV** each time $\Rightarrow$ different ciphertext

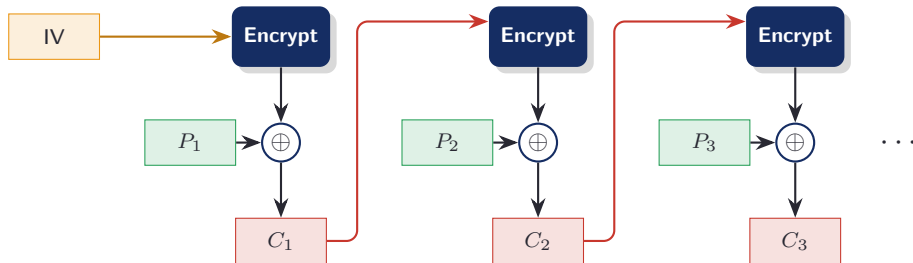
- CBC, CFB and OFB need a starting value, the **IV**, to begin the chain
- The IV must be known to **both sender and receiver**
- It does not need to be secret, but it must be **protected from change** and should be **unpredictable**
- Using a fresh IV means encrypting the same message twice gives **different ciphertexts**

OFB warning

Never reuse the same key and IV in OFB: it would repeat the key stream.



Cipher Feedback (CFB) Mode



How it works

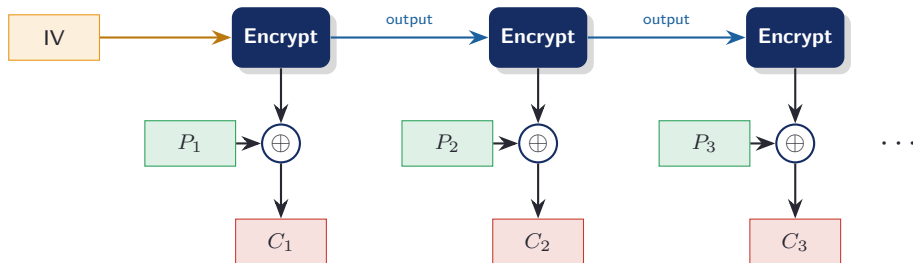
The **previous ciphertext** is encrypted, and the result is XORed with the plaintext:

$C_i = P_i \oplus E_K(C_{i-1})$. Can work on small units (e.g. 8 bits) at a time.

Key property

Turns a block cipher into a **self-synchronizing stream cipher**. Decryption uses the **encrypt** function too. Good for character-by-character data.

Output Feedback (OFB) Mode



How it works

Like CFB, but the **output of the encryption function** (not the ciphertext) is fed back. The key stream does not depend on the message.

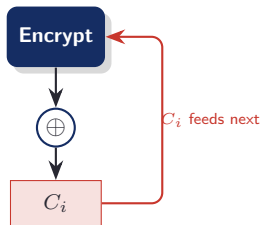
Key property

A **bit error** in one ciphertext block affects **only that bit** of plaintext; errors do not spread. Good for noisy channels such as satellite links.

CFB vs. OFB: What Is Fed Back?

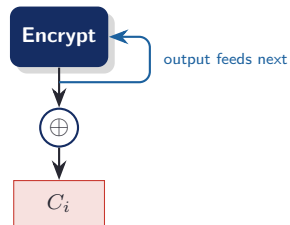


CFB: ciphertext is fed back



An error in C_i also garbles the **next** block(s).

OFB: cipher output is fed back



An error in C_i affects **only** that block. But OFB is more open to message-stream **modification**.

Comparing the Four Modes



Mode	Description	Same $P \Rightarrow$ same C ?	Typical use
ECB	Each block encrypted independently	Yes	Short data, e.g. a key
CBC	XOR with previous ciphertext, then encrypt	No	Bulk data, authentication
CFB	Encrypt previous ciphertext, XOR with plaintext	No	Stream data, characters
OFB	Encrypt previous cipher output, XOR with plaintext	No	Noisy channels (satellite)

Quick Check



Q1

How many sub-keys does IDEA use in total?

$8 \times 6 + 4 = 52$

Q2

What is 65537 in IDEA?

Multiply* modulus $2^{16}+1$

Q3

How many rounds for AES-128?

10 rounds

Q4

Which AES step uses the key?

Only **AddRoundKey**

Q5

Which mode leaks repeated blocks?

ECB



Lesson 13: IDEA

- 64-bit block, 128-bit key
- XOR, Add*, Multiply*
- $8 \text{ rounds} \times 6 + 4 = 52$ sub-keys
- 25-bit key shifting
- Output transformation
- Inverse sub-keys decrypt

Lesson 14: AES

- 128-bit block; 10/12/14 rounds
- SubBytes, ShiftRows
- MixColumns, AddRoundKey
- Key expansion: RotWord, SubWord, Rcon
- Equivalent inverse cipher

Lesson 15: Modes

- ECB: independent blocks
- CBC: chain with IV
- CFB: ciphertext feedback
- OFB: output feedback

Review questions

1. Give a brief account of IDEA.
2. Explain the structure of AES and its four transformations.
3. Explain AES key expansion.
4. Explain the ECB, CBC, CFB and OFB modes with diagrams.



- ① William Stallings, *Cryptography and Network Security*, PHI Publishers.
- ② Charlie Kaufman, Radia Perlman, *Network Security*, Pearson Education.
- ③ Bruce Schneier, *Applied Cryptography*, John Wiley & Sons.
- ④ Roberta Bragg, *Network Security: The Complete Reference*, TMH.

Thank You! Questions?